PKCERT AI & Software Development Internship
Task 05: Data Preprocessing, Visualization & SQL

Abdullah Amir

AI and Software Development
08 July 2026

Contents

Overview

For this task I worked with the **New York City Airbnb Open Data (2019)** from Kaggle. It lists close to 49,000 Airbnb listings across the five boroughs, with a nice mix of numeric columns (price, minimum nights, number of reviews) and categorical ones (borough, neighbourhood, room type). It also has genuine missing values and some wild outliers, which makes it a realistic dataset to clean rather than a tidy toy example.

Everything below is produced by a single script, `airbnb_analysis.py`, which runs in four stages: it cleans the raw CSV, draws five figures, builds encoded and scaled feature tables, and finally loads the clean data into a small SQLite database and queries it from Python. The dataset has 16 columns and 48,895 rows to start with.

Part A: Data Cleaning and Preprocessing

Missing Values

The first thing I checked was where the gaps were. Four columns had missing values: `name` (16), `host_name` (21), and both `last_review` and `reviews_per_month` (10,052 each). The two review columns are missing together, and that is the clue to what is going on: those listings have simply never been reviewed.

```
# name / host_name: only a handful missing, fill with a placeholder
df["name"] = df["name"].fillna("Unknown")
df["host_name"] = df["host_name"].fillna("Unknown")

# reviews_per_month: missing means the listing was never reviewed,
# so the right value is 0, not the mean or median
df["reviews_per_month"] = df["reviews_per_month"].fillna(0)

```

```
# last_review: a date that only exists for reviewed listings.
# We do not use it in the numeric analysis, so we drop the column.
df = df.drop(columns=["last_review"])
```

Listing 1: Handling the missing values column by column.

The reasoning behind each choice: `name` and `host_name` are just text labels with a few gaps, so a plain "Unknown" placeholder keeps the rows without pretending we know something. For `reviews_per_month` the missing value genuinely means zero reviews per month, so filling with 0 is the honest answer. `last_review` is a date that only makes sense for listings that were reviewed, and since it plays no part in the numeric work, I dropped the whole column rather than invent dates.

Duplicate Records

Next I looked for duplicates, both fully repeated rows and repeated listing ids (since `id` should be unique per listing).

```
print(df.duplicated().sum())             # fully duplicated rows -> 0
print(df.duplicated(subset="id").sum()) # duplicated ids        -> 0
df = df.drop_duplicates()
df = df.drop_duplicates(subset="id")
```

Listing 2: Checking for duplicates.

This dataset turned out to be clean on that front: zero fully duplicated rows and zero repeated ids. I still ran the drop calls so the pipeline stays safe if the data is ever refreshed with duplicates in it.

Outliers

Price and minimum nights both had unrealistic extremes. Eleven listings had a price of 0, which cannot be a real nightly rate, so those went first. For the rest of the price column I used the classic IQR rule: anything above the upper fence (Q3 plus 1.5 times the interquartile range) is treated as an outlier.

```
# price = 0 is not a valid nightly rate
df = df[df["price"] > 0]

# IQR rule on price to trim the extreme right tail
q1, q3 = df["price"].quantile([0.25, 0.75])
upper_fence = q3 + 1.5 * (q3 - q1)       # about 334
df = df[df["price"] <= upper_fence]

# a minimum stay over a year is unrealistic for a short-term rental
df = df[df["minimum_nights"] <= 365]
```

Listing 3: Removing outliers from price and minimum nights.

The price fence came out at about $334. That flagged 2,972 listings on the high end, which I removed to stop a handful of luxury outliers from dominating every chart and average. I also dropped 13 listings that asked for a minimum stay of more than a year, since those are clearly not normal short-term rentals. After all of this the dataset went from 48,895 rows down to a clean **45,899 rows**, which I saved to `airbnb_cleaned.csv`.

Part B: Data Visualization

With clean data in hand I drew five figures using Matplotlib and Seaborn. Each one is saved into the `figures/` folder by the script.

1. Histogram: Price Distribution

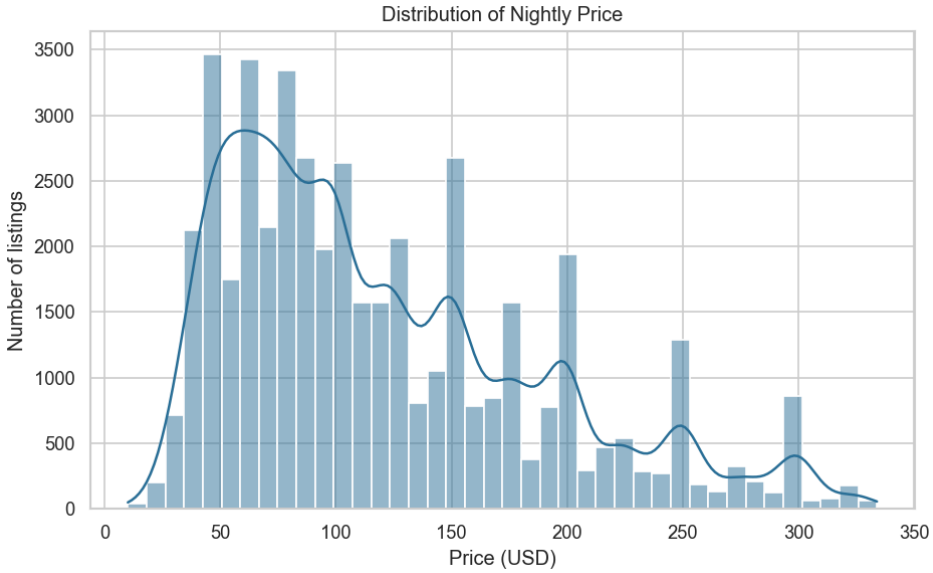

Figure 1: Distribution of nightly price after cleaning.

The price distribution is strongly right skewed. Most listings sit between roughly \$50 and \$150 a night, and the count tails off steadily after that. The little spikes at round numbers like 100, 150, and 200 are a nice human detail: hosts clearly love pricing at round figures rather than something like \$147.

2. Boxplot: Price by Room Type

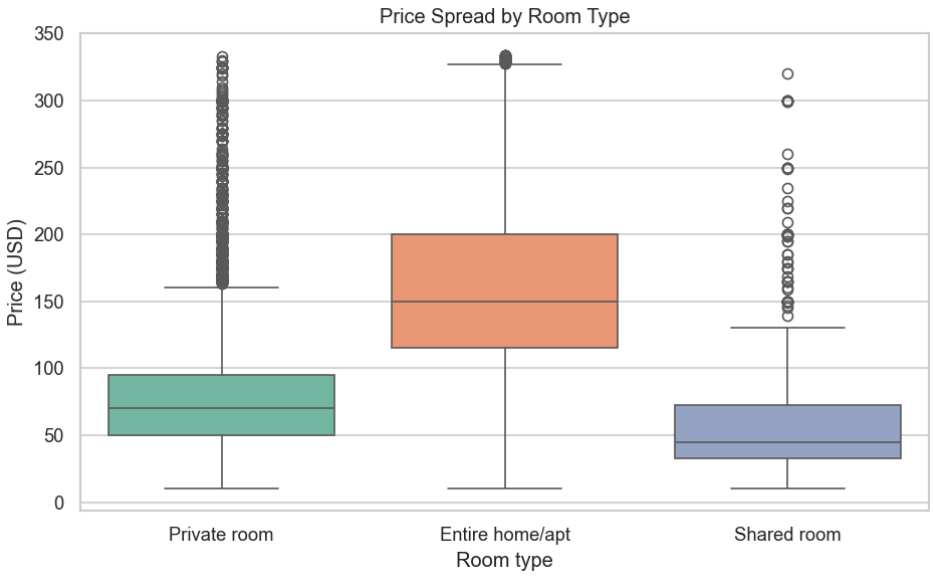

Figure 2: Price spread for each room type.

Splitting price by room type shows exactly the ordering you would expect. An entire home or apartment has the highest median (around $150) and the widest spread, a private room sits lower (around $70), and a shared room is cheapest of all. Even after trimming outliers there are still dots above the whiskers, which just means each room type has its own pricier listings relative to its group.

3. Correlation Heatmap

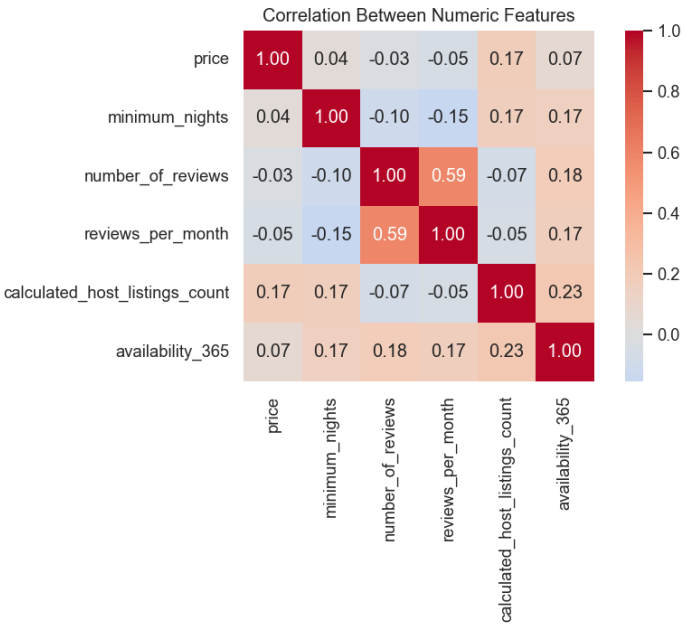

Figure 3: Correlation between the numeric features.

The heatmap looks for linear relationships between the numeric columns. The strongest one is between `number_of_reviews` and `reviews_per_month` at 0.59, which makes sense as both measure review activity. What stands out just as much is what is *not* there: price barely correlates with anything numeric (all close to 0), which tells me that price is driven mostly by the categorical features like room type and borough rather than by counts of reviews or availability.

4. Average Price by Borough

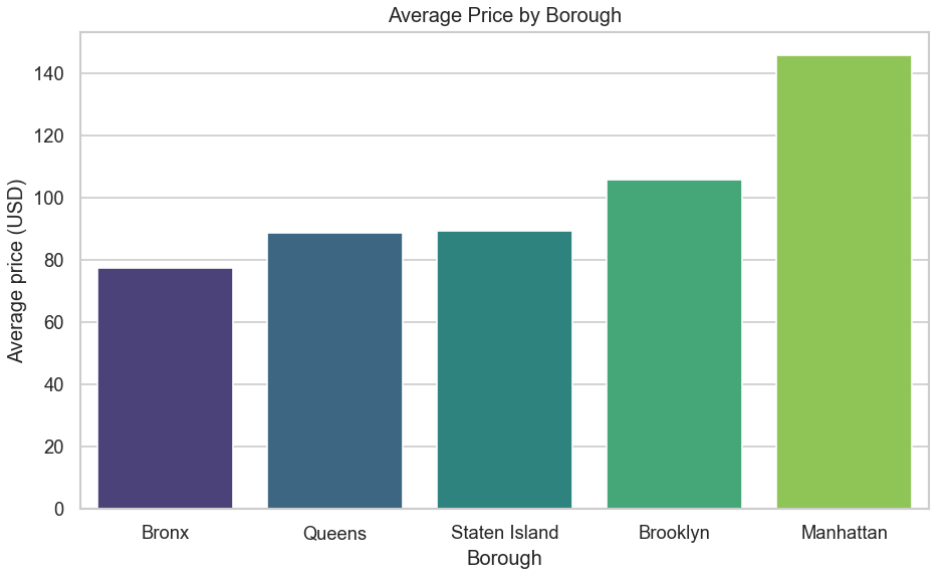

Figure 4: Average nightly price in each borough.

This bar chart confirms the point the heatmap hinted at. Manhattan is comfortably the most expensive borough on average (around $146), Brooklyn follows (around $106), and the Bronx is the cheapest (around $77). Location carries a lot of the price signal.

5. Listings Map by Borough

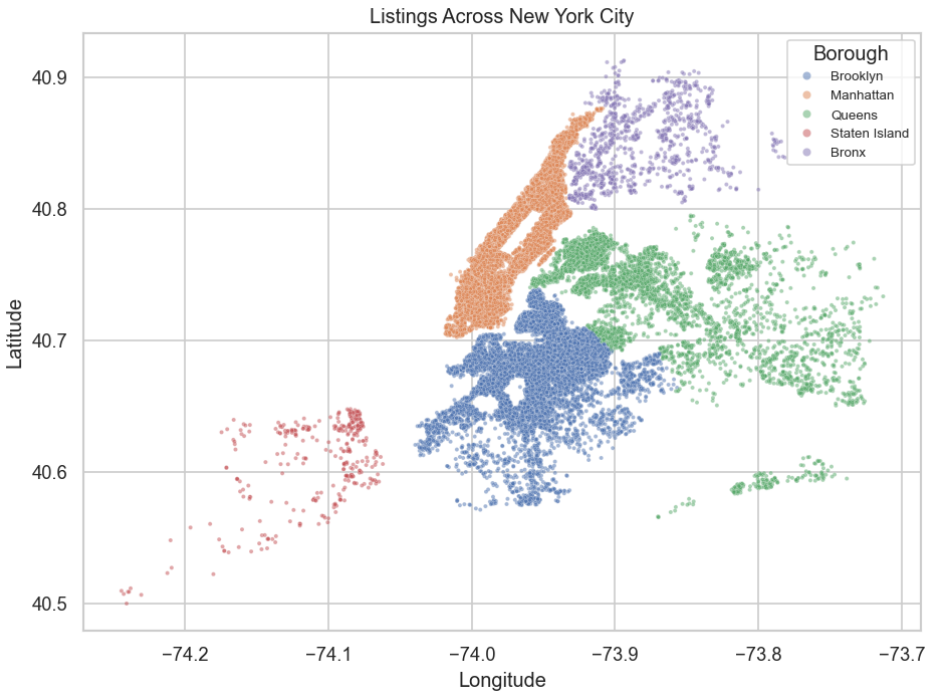

Figure 5: Every listing plotted by longitude and latitude, coloured by borough.

Plotting each listing by its longitude and latitude basically redraws the map of New York City without ever loading a map. The colours separate cleanly into the five boroughs, and you can see how densely packed Manhattan and Brooklyn are compared with the sparse scatter of Staten Island. It is a good reminder that latitude and longitude are real features, not just ID numbers.

Part C: Feature Engineering

Before any machine learning model can use this data, the text columns need to become numbers and the numeric columns need to be on a comparable scale.

Identifying and Encoding Categorical Features

The categorical columns are `name`, `host_name`, `neighbourhood_group`, `neighbourhood`, and `room_type`. The two that matter for modelling are the low-cardinality `neighbourhood_group` (5 values) and `room_type` (3 values), plus `neighbourhood` which has 219 distinct values.

```
from sklearn.preprocessing import LabelEncoder

# Label Encoding: one integer code per category.
# Good for a high-cardinality column like neighbourhood (219 values).
le = LabelEncoder()
fe["neighbourhood_encoded"] = le.fit_transform(fe["neighbourhood"])

# One-Hot Encoding: one 0/1 column per category.
# Good for low-cardinality columns, avoids a fake ordering.
fe = pd.get_dummies(fe,
        columns=["neighbourhood_group", "room_type"],
        prefix=["borough", "room"])
```

Listing 4: Label encoding and one-hot encoding.

I used the two encoders deliberately for different jobs. **Label encoding** turns each category into a single integer, which is compact and works well for `neighbourhood` where 219 one-hot columns would be wasteful. The catch is that it invents an ordering (code 5 looks "bigger" than code 2 even when the categories are not ranked). **One-hot encoding** avoids that by giving each category its own 0/1 column, so no false ordering sneaks in. That is why I used it for `neighbourhood_group` and `room_type`, which produced columns like `borough_Manhattan` and `room_Private room`.

Normalization and Standardization

The numeric columns live on very different scales: price runs into the hundreds while `reviews_per_month` is usually near 1. If you feed those straight into a distance-based or gradient-based model, the large-valued columns drown out the small ones. Standardization fixes this by rescaling each column to a mean of 0 and a standard deviation of 1.

```
from sklearn.preprocessing import StandardScaler

num_cols = ["price", "minimum_nights", "number_of_reviews",
            "reviews_per_month", "calculated_host_listings_count",
            "availability_365"]
scaler = StandardScaler()
scaled = scaler.fit_transform(df[num_cols])
# every scaled column now has mean ~0 and std ~1
```

Listing 5: Standardizing the numeric features.

After scaling, every one of those columns has a mean of essentially 0 and a standard deviation of 1, so they all contribute on equal footing. This matters for models like k-nearest neighbours, SVMs, and anything using gradient descent, where features on a huge scale would otherwise dominate the result purely because of their units. The engineered table is saved to `airbnb_features.csv`.

Part D: SQL Fundamentals with Python

For the final part I moved the clean data into a real relational database. I used SQLite through Python's built-in `sqlite3` module, which needs no server and stores the whole database in one file.

Building the Database

I created two tables: a `listings` table from the cleaned data, and a small `boroughs` dimension table that maps each borough to a short code. Having a second table gives us something real to JOIN on.

```
import sqlite3
conn = sqlite3.connect("airbnb.db")

# main fact table straight from the cleaned DataFrame
listings.to_sql("listings", conn, if_exists="replace", index=False)

# small dimension table to join against
boroughs = pd.DataFrame({
```

```
    "borough_name": ["Manhattan", "Brooklyn", "Queens",
                     "Bronx", "Staten Island"],
    "borough_code": ["MN", "BK", "QN", "BX", "SI"],
    "is_island":    [1, 0, 0, 0, 1],
})
boroughs.to_sql("boroughs", conn, if_exists="replace", index=False)
```

Listing 6: Creating the SQLite database from the cleaned DataFrame.

Running the Four Query Types

With the tables in place I ran the four query types the task asks for, reading each result straight back into Pandas with `pd.read_sql_query`.

SELECT and WHERE. A basic `SELECT` pulls the first few listings, and a `WHERE` clause filters to a specific slice, here the cheap entire homes in Manhattan under $100 a night.

```
SELECT name, price, room_type
FROM listings
WHERE neighbourhood_group = 'Manhattan'
  AND room_type = 'Entire home/apt'
  AND price < 100
LIMIT 5;
```

Listing 7: A filtered query.

GROUP BY. Grouping by room type and aggregating gives a compact summary of the whole table:

```
SELECT room_type,
       COUNT(*)             AS listings,
       ROUND(AVG(price),2) AS avg_price
FROM listings
GROUP BY room_type
ORDER BY avg_price DESC;
```

Listing 8: Aggregating with GROUP BY.

room_type	listings	avg_price
Entire home/apt	22,779	162.55
Private room	21,985	79.05
Shared room	1,135	59.35

Table 1: Result of the GROUP BY query.

JOIN. Finally, a `JOIN` stitches the `listings` table to the `boroughs` table on the borough name, so each aggregated row also carries its short borough code.

```
SELECT b.borough_code,
       l.neighbourhood_group    AS borough,
       COUNT(*)                 AS listings,
       ROUND(AVG(l.price),2)    AS avg_price
FROM listings l
JOIN boroughs b
  ON l.neighbourhood_group = b.borough_name
GROUP BY b.borough_code, l.neighbourhood_group
```

```
ORDER BY avg_price DESC;
```

Listing 9: Joining the two tables.

code	borough	listings	avg_price
MN	Manhattan	19,500	145.96
BK	Brooklyn	19,400	105.76
SI	Staten Island	365	89.24
QN	Queens	5,565	88.88
BX	Bronx	1,069	77.44

Table 2: Result of the JOIN query.

The join result lines up neatly with the bar chart from Part B, which is a good sanity check that the SQL side and the Pandas side tell the same story. Python connects to the database, runs each query, and prints the results, so the full loop from raw CSV to queryable database is covered.

Conclusion

This task walked through the whole early pipeline of a data project on a real dataset. Part A was about making messy data trustworthy: filling missing values in a way that respects what they mean, checking for duplicates, and trimming outliers with the IQR rule. Part B turned the clean data into five figures that each said something useful, from the round-number pricing habit to the map that drew itself out of latitude and longitude. Part C prepared the data for modelling with label and one-hot encoding and standardization, and Part D showed that the same cleaned data sits comfortably in a relational database where SQL can slice it with SELECT, WHERE, GROUP BY, and JOIN. Together these are the day-to-day steps that come before any model is ever trained.

The full project, including the analysis script, the cleaned dataset, the figures, and this report, is uploaded to my GitHub repository for the internship: https://github.com/AbdullahAmir06/ai-internship-abdullahamir.